

Python Data Visualization Library Documentation Guide

Matplotlib & Seaborn

ShadowFox Data Science Internship — Beginner Task

This documentation guide covers two of Python's most powerful visualization libraries — **Matplotlib** and **Seaborn**. It includes a library overview, detailed graph types with code snippets, and a comparison to help you choose the right tool for any task.

Table of Contents

Section	Content
1. Matplotlib Overview	Introduction, anatomy, and key features
2. Matplotlib — Graph Types	Line, Bar, Histogram, Scatter, Pie, Area
3. Seaborn Overview	Introduction and categories
4. Seaborn — Graph Types	Scatter, Line, Bar, Count, Box, Heatmap, KDE, Reg
5. Comparison	Ease of use, customization, interactivity, performance

1. Matplotlib — Library Overview

Matplotlib is Python's foundational plotting library, introduced in **2002 by John Hunter**. It provides a MATLAB-like interface for creating static, animated, and interactive visualizations. Built on NumPy arrays, it gives developers fine-grained control over every element of a figure.

Core Anatomy of a Matplotlib Figure

Component	Description
Figure	The top-level container that holds all plot elements. Can contain one or multiple Axes.
Axes	The actual plot area where data is drawn. Each Axes has x/y (and optionally z) axes.
Axis	The number line — controls tick marks, tick labels, and axis limits.
Artists	Everything visible on the figure: lines, text, patches, images, etc.

Key Features

- Supports 2D and basic 3D plotting
- Fine-grained control over colors, line styles, markers, fonts, and annotations
- Integrates seamlessly with NumPy, Pandas, and SciPy
- Exports to PNG, PDF, SVG, EPS, and more
- Supports interactive backends (Qt, Tk, WebAgg)
- Pyplot sub-library provides a simple MATLAB-like interface

Standard Imports

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

2. Matplotlib — Graph Types

Line Chart

Function: `plt.plot()`

Description: Connects data points with a continuous line. Best for showing trends over time or continuous data.

Use Case: Time series, stock prices, temperature trends

Code Example:

```
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.figure(figsize=(8, 4))
plt.plot(x, y, color='steelblue', linewidth=2, label='sin(x)')
plt.title('Line Chart – Sine Wave')
plt.xlabel('X Axis'); plt.ylabel('Y Axis')
plt.legend(); plt.grid(True, alpha=0.3)
plt.show()
```

Bar Chart

Function: `plt.bar()`

Description: Uses rectangular bars whose height represents the frequency or value of a category.

Use Case: Comparing values across categories, e.g., monthly sales, team scores

Code Example:

```
categories = ['A', 'B', 'C', 'D', 'E']
values     = [23, 45, 12, 67, 34]
plt.figure(figsize=(8, 4))
plt.bar(categories, values, color='steelblue', edgecolor='navy', width=0.5)
plt.title('Bar Chart'); plt.xlabel('Category'); plt.ylabel('Value')
plt.show()
```

Histogram

Function: `plt.hist()`

Description: Divides continuous data into bins and shows the frequency distribution.

Use Case: Analysing the distribution of scores, salaries, or any continuous variable

Code Example:

```
data = np.random.normal(loc=0, scale=1, size=1000)
plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, color='cornflowerblue',
         edgecolor='white', alpha=0.8)
plt.title('Histogram – Normal Distribution')
plt.xlabel('Value'); plt.ylabel('Frequency')
plt.show()
```

Scatter Plot

Function: `plt.scatter()`

Description: Uses dots to show the relationship or correlation between two numeric variables.

Use Case: Visualising correlations, clustering, outlier detection

Code Example:

```
x1 = np.random.rand(50); y1 = x1 + np.random.rand(50)*0.3
x2 = np.random.rand(50); y2 = x2**2
plt.figure(figsize=(8, 4))
plt.scatter(x1, y1, c='steelblue', label='Dataset 1', alpha=0.7)
plt.scatter(x2, y2, c='tomato', label='Dataset 2', alpha=0.7)
plt.title('Scatter Plot'); plt.legend(); plt.show()
```

Pie Chart

Function: `plt.pie()`

Description: Shows proportions of a whole using wedge-shaped slices.

Use Case: Market share, budget allocation, survey responses

Code Example:

```
labels = ['Python', 'Java', 'C++', 'JavaScript', 'Other']
sizes = [35, 25, 20, 15, 5]
colors = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B2']
plt.figure(figsize=(7, 7))
plt.pie(sizes, labels=labels, colors=colors,
        autopct='%1.1f%%', startangle=90,
        wedgeprops={'edgecolor': 'white', 'linewidth': 2})
plt.title('Pie Chart – Language Popularity')
plt.show()
```

Area Chart

Function: `plt.fill_between()` / `plt.stackplot()`

Description: Shades the area under a line to emphasise volume or cumulative quantity.

Use Case: Stacked contributions over time, resource utilisation

Code Example:

```
x = np.arange(1, 11)
y1 = np.array([3, 4, 3, 5, 6, 5, 7, 8, 7, 9])
y2 = np.array([1, 2, 2, 3, 4, 3, 4, 5, 4, 5])
plt.figure(figsize=(8, 4))
plt.stackplot(x, y1, y2, labels=['Series A', 'Series B'],
              colors=['#4C72B0', '#DD8452'], alpha=0.8)
plt.title('Stacked Area Chart'); plt.legend(loc='upper left')
plt.xlabel('Month'); plt.ylabel('Value'); plt.show()
```

3. Seaborn — Library Overview

Seaborn is a high-level statistical visualization library **built on top of Matplotlib**. It simplifies the creation of attractive, informative plots with clean default themes and built-in support for Pandas DataFrames. It is especially powerful for exploratory data analysis (EDA).

Plot Categories in Seaborn

Category	Functions
Relational	scatterplot(), lineplot(), relplot()
Categorical	barplot(), countplot(), boxplot(), violinplot(), swarmplot(), catplot()
Distribution	histplot(), kdeplot(), rugplot(), displot()
Regression	regplot(), lmplot()
Matrix	heatmap(), clustermap()

Key Features

- Automatic integration with Pandas DataFrames — pass column names directly
- Built-in themes: darkgrid, whitegrid, dark, white, ticks
- Color palettes: deep, muted, Set1, Set2, husl, coolwarm, and more
- hue, size, and style parameters for multi-dimensional encoding
- FacetGrid for multi-panel plots with one line of code
- Statistical annotations (confidence intervals, regression lines) built in

Standard Imports

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme(style='whitegrid') # Apply a clean default theme
```

4. Seaborn — Graph Types

Scatter Plot

Function: `sns.scatterplot()`

Description: Shows the relationship between two numeric variables. The hue parameter automatically colour-codes a third categorical variable.

Use Case: Correlation analysis, outlier detection, group comparisons

Code Example:

```
tips = sns.load_dataset('tips')
plt.figure(figsize=(8, 5))
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='sex', style='time', size='size')
plt.title('Total Bill vs Tip (by gender)')
plt.show()
```

Line Plot

Function: `sns.lineplot()`

Description: Plots a line with optional confidence interval shading. Ideal for time-series data grouped by a category.

Use Case: Trends over time, A/B experiment results

Code Example:

```
flights = sns.load_dataset('flights')
plt.figure(figsize=(10, 5))
sns.lineplot(data=flights, x='year', y='passengers', hue='month')
plt.title('Monthly Passengers Over Years')
plt.show()
```

Bar Plot

Function: `sns.barplot()`

Description: Shows the mean (or another estimator) of a numeric variable for each category, with 95% confidence intervals.

Use Case: Comparing group means, e.g., average tip by day of week

Code Example:

```
tips = sns.load_dataset('tips')
plt.figure(figsize=(8, 5))
sns.barplot(data=tips, x='day', y='total_bill',
            hue='sex', palette='Set2')
plt.title('Average Total Bill by Day and Gender')
plt.show()
```

Count Plot

Function: `sns.countplot()`

Description: Displays the count of observations in each category — equivalent to a bar plot of categorical frequencies.

Use Case: Class distribution, survey response counts

Code Example:

```
titanic = sns.load_dataset('titanic')
plt.figure(figsize=(8, 5))
sns.countplot(data=titanic, x='class',
              hue='survived', palette='Set1')
plt.title('Passenger Class vs Survival Count')
plt.show()
```

Box Plot

Function: `sns.boxplot()`

Description: Visualises the five-number summary (min, Q1, median, Q3, max). Points beyond 1.5 x IQR are shown as outliers.

Use Case: Comparing distributions, detecting outliers, quality control

Code Example:

```
iris = sns.load_dataset('iris')
plt.figure(figsize=(9, 5))
sns.boxplot(data=iris, x='species', y='sepal_length',
            palette='pastel')
plt.title('Sepal Length Distribution by Species')
plt.show()
```

KDE Plot

Function: `sns.kdeplot()`

Description: Kernel Density Estimate — a smoothed, continuous version of a histogram showing the probability density of a variable.

Use Case: Distribution comparison between multiple groups

Code Example:

```
tips = sns.load_dataset('tips')
plt.figure(figsize=(8, 5))
sns.kdeplot(data=tips, x='total_bill', hue='time',
            fill=True, alpha=0.4, palette='Set2')
plt.title('KDE - Total Bill by Meal Time')
plt.show()
```

Heatmap

Function: `sns.heatmap()`

Description: Encodes a matrix of numeric values using colour intensity. Brighter colours indicate higher values. Commonly used with correlation matrices.

Use Case: Feature correlation in ML, confusion matrices, pivot table summaries

Code Example:

```
flights = sns.load_dataset('flights')
pivot = flights.pivot_table(index='month',
                             columns='year', values='passengers')
plt.figure(figsize=(12, 7))
sns.heatmap(pivot, annot=True, fmt='d',
            cmap='YlOrRd', linewidths=0.5)
plt.title('Heatmap – Monthly Passengers')
plt.show()
```

Regression Plot

Function: `sns.regplot()`

Description: Draws a scatter plot with a best-fit regression line and 95% confidence interval band.

Use Case: Exploring linear relationships, trend analysis

Code Example:

```
tips = sns.load_dataset('tips')
plt.figure(figsize=(8, 5))
sns.regplot(data=tips, x='total_bill', y='tip',
            scatter_kws={'alpha':0.5}, line_kws={'color':'red'})
plt.title('Regression – Total Bill vs Tip')
plt.show()
```


5. Comparison — Matplotlib vs Seaborn

Choosing between Matplotlib and Seaborn depends on your task. The table below summarises the key differences across four dimensions.

Dimension	Matplotlib	Seaborn
Ease of Use	Steeper learning curve. Requires explicit configuration of many elements.	Beginner-friendly. Consistent defaults and concise syntax.
Customisation	Maximum control — every element (color, font, etc.) is configurable.	High-level styling is quick; deeper customisation easy; deep customisation possible.
Interactivity	Static by default; requires backends like Agg for non-interactive plots.	Also static by default, but Matplotlib's interactive backends are more accessible.
Statistical Plots	Possible but requires manual calculation of statistics and error bars.	Built-in statistical aggregation (mean, confidence intervals, etc.).
Pandas Integration	Works with Pandas but requires explicit data extraction.	Seamless extraction of a DataFrame and column names.
Large Datasets	Performs well; vector rendering can slow down with millions of points.	Similar performance, but abstractions add minor overhead.
Best For	Publication-quality figures, fine-tuned customisation.	EDA, statistical visualisation, quick multi-group comparison.

When to use Matplotlib

- You need complete control over every visual detail
- Creating publication-ready or presentation-quality figures
- Building custom or non-standard chart types
- 3D visualisation with `mpl_toolkits`

When to use Seaborn

- Quick exploratory data analysis (EDA) on Pandas DataFrames
- Statistical plots with confidence intervals out of the box
- Multi-group comparisons using hue, style, and size
- Aesthetically pleasing plots with minimal code

Tip: You don't have to choose! Most professional workflows combine both — use Seaborn for rapid EDA and statistical insight, then fine-tune with Matplotlib for final presentation.